ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

ĐỒ ÁN THIẾT KẾ LUẬN LÝ

HK251

Giảng viên hướng dẫn: Tôn Huỳnh Long
Sinh viên thực hiện: Bùi Thanh Quí - 2312865
Lê Ngọc Trường - 2313705
Đỗ Đình Thạch - 2213172

THÀNH PHỐ HỒ CHÍ MINH - 1/2026

Contents

1. Giới thiệu đề tài 3
1.1. Bối cảnh và động lực 3
1.2. Mục tiêu của đồ án 3
1.3. Phạm vi đề tài 4
2. Cơ sở lí thuyết 5
2.1. Kiến trúc RISC & MIPS 5
2.1.1. Tổng quan về RISC 5
2.1.2. Kiến trúc MIPS 6
2.1.3. Đặc tả kiến trúc phần cứng của hệ thống 6
2.1.4. Định dạng tập lệnh 6
2.2. Kỹ thuật Pipeline 7
2.2.1. Khái niệm chung 7
2.2.2. Mô hình Pipeline 5 tầng (5-Stage Pipeline) 7
2.3. Các vấn đề trong Pipeline và giải pháp 8
2.3.1. Xung đột dữ liệu (Data Hazard) 8
2.3.2. Xung đột nạp - sử dụng (Load-Use Hazard) 9
2.3.3. Xung đột điều khiển (Control Hazard) 9
3. Thiết kế hệ thống 9
3.1. Sơ đồ khối tổng quát 9
3.1.1. Sơ đồ mức đỉnh (Top-level Module) 10
3.1.2. Kiến trúc nội bộ và Luồng dữ liệu 10
3.1.3. Các khối xử lý xung đột 11
3.2. Thiết kế Datapath (Đường dữ liệu) 11
3.2.1. Giai đoạn 1: Nạp lệnh (Instruction Fetch - IF) 11
3.2.2. Giai đoạn 2: Giải mã (Instruction Decode - ID) 12
3.2.3. Giai đoạn 3: Thực thi (Execute - EX) 12
3.2.4. Giai đoạn 4: Truy xuất bộ nhớ (Memory Access - MEM) 13
3.2.5. Giai đoạn 5: Ghi hồi (Write Back - WB) 13
3.3. Thiết kế Control Unit (Khối điều khiển) 13
3.3.1. Sơ đồ tín hiệu vào/ra 13
3.3.2. Nguyên lý hoạt động và Giải mã lệnh 14
3.3.3. Bảng tóm tắt tín hiệu điều khiển 15
4. Hiện thực (Implementation) 15
4.1. Các module cơ bản 15
4.1.1. Bộ làm toán (ALU - Arithmetic Logic Unit) 15
4.1.2. Tập thanh ghi (Register File) 16
4.1.3. Bộ mở rộng dấu (Sign Extend) 16
4.1.4. Bộ nhớ lệnh (Instruction Memory) 17
4.1.5. Bộ nhớ dữ liệu (Data Memory) 17
4.2. Hiện thực Pipeline & Registers 17
4.2.1. Kết nối Datapath (Top-level Datapath) 17
4.2.2. Các thanh ghi đường ống (Pipeline Registers) 18

4.3. Xử lý xung đột 19
4.3.1. Xử lý Xung đột dữ liệu (Data Hazard) - Kỹ thuật Forwarding 19
4.3.2. Xử lý Xung đột Nạp-Dùng (Load-Use Hazard) - Kỹ thuật Stall 19
4.3.3. Xử lý xung đột Điều khiển (Control Hazard) - Kỹ thuật Flush 20
5. Kiểm thử và đánh giá 20
5.1. Kịch bản kiểm thử 21
5.2. Kết quả mô phỏng 22
5.3. Kết quả tổng hợp và đánh giá tài nguyên 25
6. **Kết luận** 26
7. **Tài liệu tham khảo** 27

Report

1. Giới thiệu đề tài

1.1. Bối cảnh và động lực

Trong kỷ nguyên công nghệ số hiện nay, bộ vi xử lý (Microprocessor/CPU) đóng vai trò là "bộ não" của hầu hết các hệ thống điện tử, từ các thiết bị di động nhỏ gọn, hệ thống nhúng (Embedded Systems) trong công nghiệp ô tô, y tế, cho đến các siêu máy tính phức tạp. Việc nắm vững kiến thức về kiến trúc máy tính và kỹ năng thiết kế vi mạch số bằng ngôn ngữ mô tả phần cứng (HDL) là yêu cầu cốt lõi đối với kỹ sư Kỹ thuật Máy tính.

Kiến trúc tập lệnh MIPS (Microprocessor without Interlocked Pipelined Stages) là một trong những kiến trúc RISC (Reduced Instruction Set Computer) kinh điển, được sử dụng rộng rãi trong giáo dục và công nghiệp nhờ vào sự hiệu quả. Tuy nhiên, các thiết kế MIPS cơ bản dạng đơn chu kỳ (Single-cycle) thường gặp hạn chế lớn về hiệu năng do chu kỳ xung nhịp bị giới hạn bởi lệnh có thời gian thực thi dài nhất.

Động lực chính để thực hiện đồ án này xuất phát từ nhu cầu:

1. **Thu hẹp khoảng cách giữa lý thuyết và thực tiễn:** Chuyển hóa các kiến thức lý thuyết về mạch số và kiến trúc máy tính thành một sản phẩm phần cứng cụ thể, hoạt động được.
2. **Tối ưu hóa hiệu năng xử lý:** Vượt qua giới hạn của kiến trúc đơn chu kỳ bằng cách áp dụng kỹ thuật đường ống (Pipelining) – một kỹ thuật then chốt trong các CPU hiện đại giúp tăng thông lượng xử lý lệnh (Instruction Throughput) bằng cách thực thi song song nhiều lệnh ở các giai đoạn khác nhau.
3. **Làm chủ kỹ thuật xử lý xung đột:** Hiểu sâu và giải quyết các bài toán hóc búa trong thiết kế vi mạch như xung đột dữ liệu (Data Hazard) và xung đột điều khiển (Control Hazard) thông qua các giải pháp phần cứng (Forwarding, Stall, Flush).

1.2. Mục tiêu của đồ án

Mục tiêu chính của đồ án là nghiên cứu, thiết kế và hiện thực hoàn chỉnh một bộ vi xử lý (CPU) 16-bit dựa trên kiến trúc RISC (Reduced Instruction Set Computer), cụ thể là mô phỏng theo tập lệnh MIPS. Đồ án tập trung vào việc giải quyết bài toán hiệu năng xử lý thông qua kỹ thuật đường ống (Pipelining) và xử lý xung đột (Hazards).

Mục tiêu cụ thể được chia thành hai nhóm:

1. **Mục tiêu cơ bản (Theo yêu cầu bắt buộc)**
   - **Thiết kế tập lệnh (ISA):** Hiện thực đầy đủ lệnh cơ bản theo định dạng 16-bit, bao gồm 3 nhóm lệnh chính: R-type (tính toán thanh ghi), I-type (tính toán tức thời/lệnh rẽ nhánh/truy xuất bộ nhớ) và J-type (lệnh nhảy).

- **Xây dựng Datapath và Control Unit:** Thiết kế các khối chức năng cốt lõi như ALU (Arithmetic Logic Unit), Register File (tập thanh ghi 8 thanh ghi đa dụng và 6 thanh ghi đặc biệt), Instruction Memory và Data Memory.
- **Mô hình hóa phần cứng:** Sử dụng ngôn ngữ mô tả phần cứng Verilog HDL để mô tả hoạt động của CPU ở mức Register Transfer Level (RTL).
- **Kiểm tra tính đúng đắn:** Đảm bảo CPU thực thi chính xác các đoạn mã máy (Machine code) được biên dịch từ hợp ngữ, bao gồm các phép toán số học, logic và các lệnh điều khiển luồng chương trình cơ bản.

2. **Mục tiêu nâng cao (Phần làm thêm & Tối ưu hóa):** Đây là trọng tâm nghiên cứu của nhóm nhằm tối ưu hóa hiệu suất hoạt động của vi xử lý:
   - **Hiện thực kiến trúc Pipeline 5 tầng:** Chuyển đổi từ kiến trúc đơn chu kỳ sang kiến trúc đa chu kỳ đường ống gồm 5 giai đoạn: *Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM)*, và *Write Back (WB)* để tăng thông lượng xử lý lệnh (Instruction Throughput).
   - **Xử lý xung đột dữ liệu (Data Hazard):** Thiết kế khối Forwarding Unit để giải quyết vấn đề phụ thuộc dữ liệu (RAW - Read After Write) mà không cần chèn lệnh rỗng (NOP), đảm bảo pipeline hoạt động liên tục với hiệu suất cao nhất.
   - **Xử lý xung đột điều khiển (Control Hazard) và Load-Use:** Thiết kế khối Hazard Detection Unit để phát hiện và xử lý các trường hợp xung đột do lệnh rẽ nhánh (Branch/Jump) hoặc lệnh đọc bộ nhớ (Load) gây ra, thông qua cơ chế Stall (dừng pipeline) và Flush (xóa lệnh sai).
   - **Hoàn thiện tập lệnh phức tạp:** Hỗ trợ chính xác các lệnh đặc biệt như `jr` (Jump Register - nhảy đến địa chỉ trong thanh ghi) và `mtsr/mfsr` (thao tác với thanh ghi đặc biệt thông qua đường truyền `funct` xuyên suốt pipeline).

1.3. Phạm vi đề tài

a. Phạm vi về Kiến trúc Tập lệnh (Instruction Set Architecture - ISA)

- **Độ rộng dữ liệu:** Vi xử lý hoạt động trên nền tảng 16-bit. Độ rộng của thanh ghi, đường dữ liệu (datapath) và bộ nhớ đều được thiết kế là 16-bit.
- **Không gian địa chỉ:** Hỗ trợ không gian địa chỉ 2^{16}, cho phép truy xuất bộ nhớ với địa chỉ 16-bit.
- **Định dạng lệnh:** Hệ thống hỗ trợ 3 định dạng lệnh cơ bản của kiến trúc MIPS là R-type, I-type và J-type với độ dài lệnh cố định là 16-bit.
- **Tập thanh ghi:**
  - 8 thanh ghi đa dụng (General Purpose Registers): từ $0 đến $7. Trong đó thanh ghi $0 luôn có giá trị bằng 0.
  - 6 thanh ghi chuyên dụng (Special Registers): $$PC$ (Program Counter), $$HI$, $$LO$ (cho phép nhân/chia), $$RA$ (Return Address), $$AT$ và $$ZERO$.
- **Các nhóm lệnh hỗ trợ:**

- Nhóm lệnh số học và logic (ALU): Cộng, trừ (có dấu/không dấu), AND, OR, XOR, NOR, Shift (dịch trái/phải), Rotate (xoay trái/phải).
- Nhóm lệnh nhân chia: Hỗ trợ tính toán và lưu kết quả vào cặp thanh ghi $\$HI$ và $\$LO7$.
- Nhóm lệnh rẽ nhánh và nhảy: `bneq`, `bgtz`, `jump`, và lệnh nhảy đến địa chỉ trong thanh ghi (`jr`).
- Nhóm lệnh truy xuất bộ nhớ: `lh` (load half-word), `sh` (store half-word).
- Nhóm lệnh thao tác thanh ghi đặc biệt: `mtsr` (Move to Special Register) và `mfsr` (Move from Special Register).

b. Phạm vi về Thiết kế Phần cứng

- **Mô hình bộ nhớ:** Sử dụng kiến trúc Harvard tách biệt giữa bộ nhớ lệnh (Instruction Memory) và bộ nhớ dữ liệu (Data Memory) để tối ưu hóa băng thông cho Pipeline.
- **Kỹ thuật Pipeline:** Thiết kế CPU theo mô hình đường ống 5 tầng (5-stage Pipeline):
  1. **IF (Instruction Fetch):** Nạp lệnh từ bộ nhớ.
  2. **ID (Instruction Decode):** Giải mã lệnh và đọc thanh ghi.
  3. **EX (Execute):** Thực thi phép toán tại ALU hoặc tính toán địa chỉ.
  4. **MEM (Memory Access):** Truy cập bộ nhớ dữ liệu.
  5. **WB (Write Back):** Ghi kết quả ngược lại thanh ghi.
- **Xử lý xung đột:**
  - **Data Hazard:** Giải quyết bằng kỹ thuật Forwarding (đưa dữ liệu từ tầng MEM và WB quay lại EX) để hạn chế tối đa việc dừng pipeline.
  - **Control Hazard & Load-Use Hazard:** Sử dụng khối Hazard Detection Unit để chèn các khoảng dừng (Stall) hoặc xóa lệnh (Flush) khi cần thiết (ví dụ: khi dự đoán sai nhánh hoặc khi dữ liệu chưa kịp sẵn sàng từ bộ nhớ).

c. Công cụ và Môi trường hiện thực

- **Ngôn ngữ thiết kế:** Verilog HDL (Hardware Description Language) ở mức RTL (Register Transfer Level).
- **Công cụ mô phỏng:** Xilinx Vivado (sử dụng để biên dịch, kiểm tra cú pháp và mô phỏng dạng sóng).

2. Cơ sở lí thuyết

2.1. Kiến trúc RISC & MIPS

2.1.1. Tổng quan về RISC

RISC (Máy tính với tập lệnh giản lược) là một triết lý thiết kế vi xử lý nhằm mục đích đơn giản hóa các lệnh riêng lẻ để chúng có thể thực hiện nhanh chóng, lý tưởng nhất

là trong một chu kỳ xung nhịp.

Các đặc điểm cốt lõi của kiến trúc RISC bao gồm:

- **Tập lệnh đơn giản và cố định:** Các lệnh có độ dài bằng nhau (trong đồ án này là 16-bit), giúp việc giải mã lệnh (Decode) trở nên đơn giản và nhanh chóng hơn so với kiến trúc CISC (Complex Instruction Set Computer).
- **Kiến trúc Load/Store:** Bộ vi xử lý không thao tác trực tiếp trên bộ nhớ (trừ lệnh nạp/ghi). Mọi tính toán đều phải được thực hiện trên các thanh ghi (Registers). Điều này giúp tách biệt việc truy xuất bộ nhớ và xử lý dữ liệu, tạo điều kiện thuận lợi cho kỹ thuật Pipeline.
- **Số lượng thanh ghi lớn:** Do các phép tính tập trung vào thanh ghi, kiến trúc RISC thường cung cấp một tập thanh ghi đa dụng đủ lớn để giảm thiểu việc truy cập bộ nhớ chậm chạp.

2.1.2. Kiến trúc MIPS

MIPS (Microprocessor without Interlocked Pipelined Stages) là một trong những kiến trúc RISC tiêu biểu nhất, được sử dụng rộng rãi trong giảng dạy và công nghiệp. Kiến trúc MIPS nhấn mạnh vào hiệu năng thông qua kỹ thuật đường ống.

Trong khuôn khổ đồ án này, nhóm thiết kế một biến thể MIPS 16-bit, giữ lại các triết lý thiết kế cốt lõi của MIPS chuẩn nhưng thu gọn không gian địa chỉ và độ rộng dữ liệu để phù hợp với quy mô đồ án môn học.

2.1.3. Đặc tả kiến trúc phần cứng của hệ thống

Hệ thống được thiết kế dựa trên các thông số kỹ thuật sau:

- **Độ rộng từ (Word Size):** 16-bit.
- **Không gian địa chỉ:** 2^{16} địa chỉ, cho phép quản lý bộ nhớ lên đến 64KB.
- **Tập thanh ghi (Register File):**
  - **08 Thanh ghi đa dụng (GPR - 16 bit):** Được đánh số từ $0 đến $7. Trong đó, thanh ghi $0 (hay $\$ZERO$) được thiết kế cứng với giá trị luôn bằng 0
  - **06 Thanh ghi đặc biệt:**
    $\$PC$ (Program Counter): Quản lý địa chỉ lệnh kế tiếp.
    $\$HI, \$LO$: Lưu trữ kết quả của phép nhân (32-bit) hoặc phép chia (Thương/Dư).
    $\$RA$ (Return Address): Lưu địa chỉ trả về khi gọi hàm (giả lập).
    $\$AT$: Assembler Temporary.
    $\$SAT$: Status/Special Register (tùy ngữ cảnh sử dụng trong thiết kế mở rộng).

2.1.4. Định dạng tập lệnh

Để đơn giản hóa quá trình giải mã (Decode Stage) trong Pipeline, toàn bộ lệnh đều có độ dài cố định 16-bit và được chia thành 3 định dạng chính: R-type, I-type, và J-type.

1. **R-type (Register-type):** Dùng cho các phép toán số học và logic giữa các thanh ghi. Cấu trúc: `Opcode(4)|rs(3)|rt(3)|rd(3)|funct(3)`
   - **Opcode:** 4-bit mã lệnh chính.
   - **rs:** 3-bit địa chỉ thanh ghi nguồn 1.
   - **rt:** 3-bit địa chỉ thanh ghi nguồn 2.
   - **rd:** 3-bit địa chỉ thanh ghi đích.
   - **funct:** 3-bit mã phụ để phân biệt các phép toán cụ thể (VD: cộng, trừ, and, or) khi có cùng Opcode.
2. **I-type (Immediate-type):** Dùng cho các lệnh thao tác với hằng số, lệnh rẽ nhánh (Branch) và truy xuất bộ nhớ (Load/Store).
   Cấu trúc: `Opcode(4)|rs(3)|rt(3)|Immediate(6)`
   - **rs:** 3-bit địa chỉ thanh ghi nguồn (hoặc cơ sở).
   - **rt:** 3-bit địa chỉ thanh ghi đích (hoặc nguồn cho Store).
   - **Immediate:** 6-bit hằng số giá trị tức thời (có dấu).
3. **J-type (Jump-type):** Dùng cho các lệnh nhảy không điều kiện đến một địa chỉ xa. Cấu trúc: `Opcode(4)|Address(12)`
   - **Address:** 12-bit địa chỉ đích (thường được dịch trái hoặc ghép với PC để tạo địa chỉ vật lý đầy đủ).

Việc chuẩn hóa định dạng lệnh này giúp bộ giải mã (Control Unit) hoạt động nhanh hơn, đồng thời vị trí các trường `rs` và `rt` cố định giúp việc đọc thanh ghi có thể diễn ra song song với giải mã lệnh.

2.2. Kỹ thuật Pipeline

2.2.1. Khái niệm chung

Pipeline (Đường ống) là kỹ thuật thực hiện lệnh song song ở mức chỉ lệnh (Instruction-Level Parallelism), cho phép nhiều lệnh được thực thi cùng một lúc tại các giai đoạn khác nhau. Thay vì chờ một lệnh hoàn thành toàn bộ chu trình mới nạp lệnh tiếp theo (như kiến trúc đơn chu kỳ), bộ xử lý Pipeline chia quá trình thực thi thành các giai đoạn nhỏ độc lập.

2.2.2. Mô hình Pipeline 5 tầng (5-Stage Pipeline)

Trong đồ án này, nhóm thiết kế vi xử lý theo mô hình Pipeline 5 tầng kinh điển của MIPS. Mỗi lệnh sẽ đi qua lần lượt các giai đoạn sau:

1. **Instruction Fetch (IF) - Nạp lệnh:**
   - CPU sử dụng giá trị của thanh ghi PC (Program Counter) để truy xuất bộ nhớ lệnh (Instruction Memory) và lấy ra mã lệnh 16-bit.
   - Đồng thời, bộ cộng sẽ tính toán địa chỉ tiếp theo ($PC+2$) để chuẩn bị cho chu kỳ sau.

2. **Instruction Decode (ID) - Giải mã:**
   - Mã lệnh từ tầng IF được đưa vào bộ giải mã (Control Unit) để sinh ra các tín hiệu điều khiển.
   - Đồng thời, CPU đọc giá trị từ tập thanh ghi (Register File) dựa trên các trường địa chỉ `rs` và `rt`.
   - Khối mở rộng dấu (Sign Extend) chuyển đổi các giá trị tức thời (Immediate) từ 6-bit sang 16-bit.
3. **Execute (EX) - Thực thi:**
   - Đây là giai đoạn tính toán chính. ALU (Arithmetic Logic Unit) thực hiện các phép toán số học (cộng, trừ, nhân, chia), logic (AND, OR, XOR) hoặc dịch bit (Shift/Rotate) dựa trên các toán hạng đã chuẩn bị.
   - Đối với lệnh rẽ nhánh (Branch), địa chỉ đích và điều kiện rẽ nhánh (So sánh, Zero flag) được kiểm tra tại đây.
4. **Memory Access (MEM) - Truy xuất bộ nhớ:**
   - Nếu là lệnh nạp (`lh`) hoặc lưu (`sh`), CPU sẽ truy cập bộ nhớ dữ liệu (Data Memory).
   - Với các lệnh không truy cập bộ nhớ, kết quả từ ALU được chuyển thẳng qua giai đoạn này.
5. **Write Back (WB) - Ghi hồi:**
   - Kết quả cuối cùng (từ ALU hoặc từ bộ nhớ) được ghi ngược lại vào tập thanh ghi đích (Register File) để hoàn tất câu lệnh.

Để tách biệt các giai đoạn này, hệ thống sử dụng các thanh ghi đường ống (Pipeline Registers): `IF/ID`, `ID/EX`, `EX/MEM`, và `MEM/WB` để lưu trữ trạng thái và tín hiệu điều khiển giữa các chu kỳ xung nhịp.

2.3. Các vấn đề trong Pipeline và giải pháp

Việc chia nhỏ quá trình thực thi dẫn đến các tình huống xung đột (Hazards), nơi lệnh sau không thể thực thi ngay lập tức do phụ thuộc vào lệnh trước. Đồ án đã giải quyết 3 loại xung đột chính:

2.3.1. Xung đột dữ liệu (Data Hazard)

Xảy ra khi một lệnh cần sử dụng dữ liệu mà lệnh đi trước chưa kịp ghi vào thanh ghi (Read-After-Write dependence).

- **Vấn đề:** Lệnh `ADD $3, $1, $2` theo sau lệnh `ADDI $1`, ... Lệnh `ADD` đọc `$1` ở giai đoạn ID, trong khi ADDI chỉ ghi giá trị mới vào `$1` ở giai đoạn WB (cách nhau 3 chu kỳ). Dữ liệu đọc được là cũ và sai.
- **Giải pháp - Kỹ thuật Forwarding (Chuyển tiếp):**

- Nhóm đã thiết kế khối Forwarding Unit. Khối này giám sát các lệnh đang ở giai đoạn MEM và WB.
- Nếu phát hiện lệnh tầng EX cần dữ liệu mà lệnh trước đó đang nắm giữ, Forwarding Unit sẽ điều khiển bộ dồn kênh (MUX) để lấy dữ liệu trực tiếp từ đầu ra của ALU (tầng MEM) hoặc bộ nhớ (tầng WB) đưa ngược về đầu vào ALU, bỏ qua bước đọc thanh ghi. Giải pháp này giúp Pipeline không bị dừng (Stall).

2.3.2. Xung đột nạp - sử dụng (Load-Use Hazard)

Đây là trường hợp đặc biệt của Data Hazard mà Forwarding không giải quyết được triệt để.

- **Vấn đề:** Khi lệnh ngay sau lệnh `Load` (`lh`) cần sử dụng chính dữ liệu vừa được nạp. Dữ liệu từ bộ nhớ chỉ có sẵn ở cuối giai đoạn MEM, nhưng lệnh sau cần nó ngay ở đầu giai đoạn EX.
- **Giải pháp - Stall (Chèn khoảng dừng):**
  - Khối **Hazard Detection Unit** được thiết kế để phát hiện tình huống này (khi `ID_EX_MemRead` tích cực và thanh ghi đích trùng với nguồn lệnh đang giải mã).
  - Hệ thống sẽ thực hiện **Stall**: Giữ nguyên giá trị PC và thanh ghi IF/ID (không nạp lệnh mới), đồng thời chèn một lệnh rỗng (NOP - bong bóng) vào giai đoạn EX để trì hoãn lệnh sau 1 chu kỳ, chờ dữ liệu sẵn sàng.

2.3.3. Xung đột điều khiển (Control Hazard)

Xảy ra đối với các lệnh rẽ nhánh (`bneq`, `bgtz`) hoặc nhảy (`jump`).

- **Vấn đề:** Mặc định, Pipeline luôn nạp lệnh kế tiếp ($PC+2$). Tuy nhiên, nếu lệnh rẽ nhánh được thực hiện (Branch Taken), các lệnh đã lỡ nạp vào đường ống là sai và không được phép thực thi.
- **Giải pháp - Flush (Xóa đường ống):**
  - Trong thiết kế này, kết quả rẽ nhánh được quyết định tại giai đoạn EX (hoặc ID với lệnh Jump).
  - Khi phát hiện rẽ nhánh thành công (`branch_taken` hoặc `jump_taken`), Hazard Unit kích hoạt tín hiệu **Flush**.
  - Các thanh ghi Pipeline (`IF/ID` hoặc `ID/EX`) bị xóa (reset về 0), biến các lệnh sai đang nằm trong đường ống thành lệnh NOP, và PC được cập nhật lại theo địa chỉ đích mới.

3. Thiết kế hệ thống

3.1. Sơ đồ khối tổng quát

Hệ thống được thiết kế theo kiến trúc Harvard, tách biệt bộ nhớ lệnh (Instruction Memory) và bộ nhớ dữ liệu (Data Memory), cho phép thực hiện việc nạp lệnh và truy

xuất dữ liệu song song trong cùng một chu kỳ xung nhịp. Đây là yếu tố tiên quyết để vận hành kỹ thuật Pipeline hiệu quả.

3.1.1. Sơ đồ mức đỉnh (Top-level Module)

Ở mức trừu tượng cao nhất, CPU được coi là một hộp đen (`cpu_top`) giao tiếp với thế giới bên ngoài thông qua các tín hiệu cơ bản:

- **Inputs:**
  - `clk` (Clock): Tín hiệu xung nhịp đồng bộ hóa toàn bộ hoạt động của hệ thống.
  - `rst` (Reset): Tín hiệu khởi động lại hệ thống (Active High), đưa bộ đếm chương trình (PC) và các thanh ghi đường ống về trạng thái mặc định.
- **Outputs:**
  - Hệ thống không có cổng xuất dữ liệu trực tiếp ra thiết bị ngoại vi trong phạm vi đồ án này. Kết quả thực thi được kiểm chứng thông qua giá trị trong tập thanh ghi (Register File) và bộ nhớ dữ liệu (Data Memory) sau khi mô phỏng.

3.1.2. Kiến trúc nội bộ và Luồng dữ liệu

Sơ đồ khối chi tiết của hệ thống bao gồm hai thành phần chính tương tác chặt chẽ với nhau: **Datapath** (Đường dữ liệu) và **Control Unit** (Khối điều khiển).

Sơ đồ khối hoàn chỉnh: Link Drive

Hoạt động của CPU được chia thành 5 giai đoạn (stages) nối tiếp nhau thông qua các thanh ghi đường ống (Pipeline Registers):

1. **Giai đoạn IF (Instruction Fetch):**
   - PC Register, Instruction Memory, PC Source Mux, Add $PC+2$.
   - Chức năng: Xác định địa chỉ lệnh kế tiếp (từ PC+2 hoặc từ bộ cộng Branch/Jump) và nạp lệnh từ bộ nhớ.
2. **Giai đoạn ID (Instruction Decode):**
   - Bao gồm: Control Unit, Register File, Sign Extend, Hazard Detection Unit, Comparator (cho Branch).
   - Chức năng: Giải mã Opcode, đọc dữ liệu từ thanh ghi nguồn, mở rộng dấu cho hằng số (Immediate), và xử lý sớm điều kiện rẽ nhánh để giảm độ trễ (Branch Penalty).
3. **Giai đoạn EX (Execute):**
   - Bao gồm: ALU, Barrel Shifter (Bộ dịch bit), Mult/Div Unit (Bộ nhân/chia), Forwarding Unit.

- Chức năng: Thực hiện các phép toán số học/logic. Đặc biệt, khối Forwarding Unit tại đây sẽ điều hướng dữ liệu từ các tầng sau quay lại đầu vào ALU để giải quyết xung đột dữ liệu.

4. **Giai đoạn MEM (Memory Access):**
   - Bao gồm: Data Memory.
   - Chức năng: Đọc hoặc ghi dữ liệu vào bộ nhớ (đối với lệnh lh, sh). Các lệnh tính toán chỉ đơn giản truyền kết quả qua giai đoạn này.
5. **Giai đoạn WB (Write Back):**
   - Bao gồm: Các bộ dồn kênh (MUX) chọn dữ liệu ghi.
   - Chức năng: Chọn kết quả từ ALU, bộ nhớ hoặc các khối chức năng khác để ghi ngược lại vào Register File.

3.1.3. Các khối xử lý xung đột

Để đảm bảo tính đúng đắn của dòng dữ liệu trong Pipeline, hệ thống tích hợp thêm hai khối chức năng nâng cao (như thể hiện trong sơ đồ):

- **Hazard Detection Unit:** Nằm tại tầng ID. Khối này phát hiện xung đột nạp-sử dụng (Load-Use Hazard) và xung đột điều khiển (Control Hazard). Nó có quyền chèn lệnh NOP (bong bóng) vào đường ống hoặc giữ nguyên trạng thái PC để dừng pipeline (Stall).
- **Forwarding Unit:** Nằm tại tầng EX. Khối này so sánh địa chỉ thanh ghi nguồn của lệnh hiện tại với địa chỉ thanh ghi đích của các lệnh trước đó (đang ở MEM hoặc WB) để quyết định "đi tắt" dữ liệu, giúp CPU không phải dừng chờ đợi.

3.2. Thiết kế Datapath (Đường dữ liệu)

Datapath là phần cốt lõi của bộ vi xử lý, chịu trách nhiệm thực hiện các thao tác xử lý dữ liệu và tính toán địa chỉ. Để đáp ứng yêu cầu về hiệu năng, thiết kế Datapath được phân chia thành 5 giai đoạn độc lập (Instruction Stages), kết nối với nhau qua các thanh ghi đường ống (Pipeline Registers).

3.2.1. Giai đoạn 1: Nạp lệnh (Instruction Fetch - IF)

Giai đoạn này chịu trách nhiệm cung cấp mã lệnh cho CPU.

- **Bộ đếm chương trình (PC Register):** Là thanh ghi 16-bit lưu địa chỉ của lệnh hiện tại. Giá trị của PC được cập nhật ở sườn dương của xung clock.
- **Bộ nhớ lệnh (Instruction Memory):** Nhận địa chỉ từ PC và trả về mã lệnh 16-bit (`instr_IF`) tương ứng.
- **Bộ cộng PC (PC Adder):** Mặc định, địa chỉ lệnh kế tiếp là $PC + 2$ (do mỗi lệnh chiếm 2 byte).

- **Bộ dồn kênh chọn PC (PC Mux):** Quyết định giá trị `pc_next` dựa trên tín hiệu điều khiển:
  1. $PC+2$: Tuần tự bình thường.
  2. `Branch Target`: Địa chỉ đích khi lệnh rẽ nhánh thành công (tính tại EX).
  3. `Jump Target`: Địa chỉ đích khi gặp lệnh nhảy không điều kiện (tính tại ID).
- **Thanh ghi IF/ID:** Lưu trữ giá trị $PC+2$ và mã lệnh (`Instruction`) để chuyển sang giai đoạn sau.

3.2.2. Giai đoạn 2: Giải mã (Instruction Decode - ID)

Tại đây, lệnh được giải mã và các toán hạng được chuẩn bị.

- **Tập thanh ghi (Register File):** Có 2 cổng đọc (`ReadData1`, `ReadData2`) tương ứng với địa chỉ `rs` và `r` từ mã lệnh. Việc ghi dữ liệu (`WriteData`) được thực hiện đồng bộ ở sườn xung nhịp (thường là sườn âm hoặc nửa sau chu kỳ để tránh xung đột đọc/ghi).
- **Khối mở rộng dấu (Sign Extend):** Chuyển đổi giá trị tức thời 6-bit (Immediate) trong lệnh I-type thành 16-bit để phục vụ tính toán.
- **Xử lý Branch sớm (Early Branch):** Để giảm độ trễ rẽ nhánh (Branch Penalty), kiến trúc so sánh hai thanh ghi (cho lệnh `bneq`) hoặc kiểm tra dấu (cho lệnh `bgtz`) ngay tại giai đoạn này thay vì đợi đến EX.
- **Thanh ghi ID/EX:** Lưu trữ các tín hiệu điều khiển (`RegWrite`, `MemRead`,...) và dữ liệu (`ReadData1`, `ReadData2`, `Imm`) cho giai đoạn thực thi.

3.2.3. Giai đoạn 3: Thực thi (Execute - EX)

Đây là nơi diễn ra các phép toán số học và logic chính.

- **ALU (Arithmetic Logic Unit):** Thực hiện các phép toán dựa trên tín hiệu ALUOp:
  - Số học: Cộng, trừ, nhân, chia (có dấu/không dấu).
  - Logic: AND, OR, XOR, NOR.
  - Dịch bit: Shift Left/Right, Rotate Left/Right (được tích hợp trong logic ALU hoặc bộ Shifter riêng).
- **Thanh ghi HI/LO:** Lưu trữ kết quả 32-bit của phép nhân hoặc Thương/Dư của phép chia.
- **Khối Forwarding (Forwarding Unit):** Điều khiển các bộ MUX tại đầu vào ALU (3-to-1 MUX) để chọn toán hạng từ:
  - Tập thanh ghi (ID/EX).
  - Kết quả tính toán vừa xong từ giai đoạn MEM (Forwarding từ EX/MEM).
  - Kết quả ghi hồi từ giai đoạn WB (Forwarding từ MEM/WB).
- **Thanh ghi EX/MEM:** Lưu kết quả ALU (`alu_result`), dữ liệu cần ghi bộ nhớ (`write_data` - từ `$rt`) và địa chỉ thanh ghi đích (`write_reg`).

3.2.4. Giai đoạn 4: Truy xuất bộ nhớ (Memory Access - MEM)

Giai đoạn này tương tác với bộ nhớ dữ liệu (Data Memory).

- **Bộ nhớ dữ liệu (Data Memory):**
  - Load (`lh`): Đọc dữ liệu từ địa chỉ do ALU tính toán.
  - Store (`sh`): Ghi dữ liệu từ thanh ghi `$rt` vào bộ nhớ tại địa chỉ do ALU tính toán.

  Với các lệnh R-type không truy cập bộ nhớ, kết quả ALU được đưa thẳng qua giai đoạn này.
- **Thanh ghi MEM/WB:** Lưu trữ dữ liệu đọc từ bộ nhớ (`mem_data`) hoặc kết quả ALU để chuẩn bị ghi ngược về thanh ghi.

3.2.5. Giai đoạn 5: Ghi hồi (Write Back - WB)

Giai đoạn cuối cùng cập nhật trạng thái hệ thống.

- **Bộ dồn kênh chọn dữ liệu ghi (Write Back Mux):**
  Chọn dữ liệu để ghi vào `Register File` dựa trên tín hiệu `MemtoReg`:
  - Chọn `ALU Result` (cho lệnh R-type, I-type tính toán).
  - Chọn `Memory Data` (cho lệnh Load).
- Địa chỉ ghi (`WriteAddr`) là `$rd` (với R-type) hoặc `$rt` (với I-type), đã được xác định và truyền suốt từ giai đoạn ID.

3.3. Thiết kế Control Unit (Khối điều khiển)

Control Unit (CU) đóng vai trò là "bộ não" của vi xử lý, chịu trách nhiệm giải mã các lệnh được nạp vào và phát sinh các tín hiệu điều khiển để kích hoạt các thành phần tương ứng trong Datapath (ALU, Register File, Data Memory, Multiplexers).

3.3.1. Sơ đồ tín hiệu vào/ra

Control Unit hoạt động dựa trên logic tổ hợp thuần túy (Combinational Logic), với các tín hiệu giao tiếp như sau:

- **Inputs (Đầu vào):**
  - `opcode [3:0]`: 4-bit mã lệnh chính, xác định loại lệnh (R-type, I-type, J-type).
  - `funct [2:0]`: 3-bit mã phụ, dùng để phân biệt các phép toán cụ thể trong nhóm lệnh R-type (ALU0, ALU1, ALU2).
- **Outputs (Đầu ra - Tín hiệu điều khiển):**
  1. **Tín hiệu quản lý thanh ghi & bộ nhớ:**
     - `RegWrite`: Cho phép ghi dữ liệu vào Register File.

- `RegDst`: Chọn địa chỉ ghi là `$rt` (0) hay `$rd` (1).
- `MemRead` / `MemWrite`: Điều khiển việc đọc/ghi bộ nhớ dữ liệu.
- `MemtoReg`: Chọn dữ liệu ghi hồi là từ ALU (0) hay từ bộ nhớ (1).

2. **Tín hiệu điều khiển ALU:**
   - `ALUSrc`: Chọn toán hạng B cho ALU là thanh ghi (0) hay hằng số Immediate (1).
   - `ALUOp [4:0]`: Mã điều khiển chức năng cụ thể cho ALU (Cộng, Trừ, AND, Shift,...).
3. **Tín hiệu rẽ nhánh:**
   - `Branch`: Kích hoạt logic kiểm tra rẽ nhánh (cho `bneq`, `bgtz`).
   - `Jump`: Kích hoạt lệnh nhảy không điều kiện (`jump`).
   - `IsBGTZ`: Tín hiệu phụ để phân biệt giữa `bneq` và `bgtz`.
4. **Tín hiệu cho các lệnh đặc biệt (Extension):**
   - `HILO_WriteEn`: Cho phép ghi vào thanh ghi `$HI`, `$LO` (lệnh nhân/chia).
   - `MTSR_WriteEn` / `MFSR_ReadEn`: Điều khiển ghi/đọc thanh ghi đặc biệt.

3.3.2. Nguyên lý hoạt động và Giải mã lệnh

Thiết kế Control Unit kết hợp cả *Main Decoder* và *ALU Decoder* trong cùng một khối logic để tối ưu hóa cho kiến trúc 16-bit. Quá trình giải mã được chia thành các trường hợp dựa trên `Opcode`:

1. **Nhóm lệnh R-type (ALU0, ALU1, ALU2)**
   - **Opcode:** 0000, 0001, 0010.
   - **Đặc điểm:** Các lệnh này thực hiện tính toán trên thanh ghi.
   - **Tín hiệu:**
     - `RegWrite = 1`: Luôn ghi kết quả.
     - `RegDst = 1`: Kết quả ghi vào thanh ghi `$rd`.
     - `ALUSrc = 0`: Toán hạng thứ 2 lấy từ thanh ghi `$rt`.
   - **ALUOp:** Được xác định dựa vào 3-bit funct. Ví dụ:
     - Nếu `funct = 000` → `ALUOp = OP_ADD/ADDU`.
     - Nếu `funct = 010` → `ALUOp = OP_MULT` (và bật `HILO_WriteEn`).
     - Nếu `Opcode = 0010` (Shift) → `ALUOp` là các lệnh dịch/xoay (`SHR`, `SHL`, `ROR`, `ROL`).
2. **Nhóm lệnh I-type (Tính toán tức thời & Load/Store)**
   - **ADDI / SLTI (`0011, 0100`):**
     - `ALUSrc = 1`: Sử dụng hằng số `Immediate`.
     - `RegDst = 0`: Kết quả ghi vào `$rt`.
   - **Load / Store (`1000, 1001`):**
     - Sử dụng `ALUOp = OP_ADDR_CALC` để tính địa chỉ theo công thức đặc biệt `(rs + imm) « 1`.

  - `LH`: Bật `MemRead`, `MemtoReg`.
  - `SH`: Bật `MemWrite`.

3. **Nhóm lệnh Rẽ nhánh (Branch & Jump)**

   - **BNEQ (`0101`):** Bật `Branch = 1`, `ALUOp = OP_SUBU`. Logic rẽ nhánh sẽ kiểm tra cờ Zero từ ALU (nếu hiệu khác 0 thì nhảy).
   - **BGTZ (`0110`):** Bật `Branch = 1` và `IsBGTZ = 1`. Logic rẽ nhánh sẽ kiểm tra cờ GTZ (Greater Than Zero).
   - **JUMP (`0111`):** Bật `Jump = 1`. Địa chỉ đích được ghép từ PC và Immediate ngay tại tầng ID.

3.3.3. Bảng tóm tắt tín hiệu điều khiển

Bảng dưới đây tóm tắt trạng thái các tín hiệu quan trọng đối với một số lệnh tiêu biểu:

Instruction	Opcode	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-format	000x	1	0	0	1	0	0	0	func
ADDI	0011	0	1	0	1	0	0	0	ADD
LH	1000	0	1	1	1	1	0	0	ADDR
SH	1001	x	1	x	0	0	1	0	ADDR
BNEQ	0101	x	0	x	0	0	0	1	SUB
JUMP	0111	x	x	x	0	0	0	0	xxx

(Lưu ý: x biểu thị giá trị don't care)

Việc thiết kế Control Unit tập trung như trên giúp đơn giản hóa sơ đồ mạch, đồng thời đảm bảo tính chính xác khi tích hợp vào hệ thống Pipeline, nơi các tín hiệu điều khiển này sẽ được truyền qua các thanh ghi đệm (`ID/EX`, `EX/MEM`, `MEM/WB`) để đến đúng giai đoạn cần thiết.

4. Hiện thực (Implementation)

Hệ thống được thiết kế và hiện thực bằng ngôn ngữ Verilog HDL. Mã nguồn được tổ chức theo mô hình phân cấp (Hierarchical Design), với module đỉnh là `cpu_top` chứa `datapath`, và `datapath` chứa các module chức năng con.

4.1. Các module cơ bản

Các module cơ bản đóng vai trò là những viên gạch nền tảng, thực hiện các chức năng chuyên biệt cụ thể. Dưới đây là mô tả chi tiết về hiện thực của từng khối:

4.1.1. Bộ làm toán (ALU - Arithmetic Logic Unit)

- **Tên file:** `alu.v`
- **Chức năng:** Thực hiện toàn bộ các phép toán số học, logic và tính toán địa chỉ.

- **Đặc điểm hiện thực:**
  - Sử dụng cấu trúc case để lựa chọn phép toán dựa trên tín hiệu điều khiển ALUOp 5-bit.
  - Hỗ trợ đầy đủ các nhóm lệnh:
    Số học: Cộng/Trừ có dấu và không dấu (`ADD`, `SUB`, `ADDU`, `SUBU`).
    Logic: `AND`, `OR`, `NOR`, `XOR`.
    Dịch/Xoay bit: `SHL`, `SHR`, `ROL`, `ROR` (thực hiện dịch bit dựa trên 4 bit thấp của toán hạng A).
    So sánh: `SLT`, `SLTU`, `SEQ` (trả về 1 hoặc 0).
    Nhân/Chia: `MULT`, `DIV` (kết quả trả về cặp thanh ghi `HI_Out` và `LO_Out`).
  - **Tính năng đặc biệt:** Đã hiện thực phép tính địa chỉ bộ nhớ chuyên biệt cho lệnh `LH/SH` theo công thức đề bài: `Result = ((rs » 1) + imm) « 1`.

4.1.2. Tập thanh ghi (Register File)

- **Tên file:** `register_file.v`
- **Chức năng:** Lưu trữ trạng thái của các thanh ghi đa dụng và chuyên dụng.
- **Đặc điểm hiện thực:**
  - **Mảng thanh ghi:** `reg [15:0] registers [0:7]`.
  - **Cơ chế đọc:** Đọc không đồng bộ (Asynchronous Read). Dữ liệu tại cổng `ReadData` có giá trị ngay khi địa chỉ `ReadAddr` thay đổi.
  - **Cơ chế ghi:** Ghi đồng bộ (Synchronous Write) tại sườn dương xung nhịp `clk` khi tín hiệu `WriteEn` tích cực.
  - **Thanh ghi đặc biệt:** Module cũng quản lý việc ghi vào các thanh ghi `$HI`, `$LO` và các thanh ghi đặc biệt khác (`$AT`, `$RA`) thông qua các tín hiệu điều khiển `HILO_WriteEn`, `MTSR_WriteEn`.
  - Ràng buộc: Thanh ghi $0 được thiết kế chỉ đọc (Read-only) với giá trị luôn bằng 0.

4.1.3. Bộ mở rộng dấu (Sign Extend)

- **Tên file:** `sign_extend.v`
- **Chức năng:** Chuyển đổi giá trị tức thời (Immediate) 6-bit từ lệnh I-type thành 16-bit để thực hiện tính toán với thanh ghi.
- **Logic:** Sao chép bit dấu (bit cao nhất của input) vào toàn bộ các bit cao của output.
  - Input: `[5:0]`
  - Output: `10Imm_6bit[5], Imm_6bit`.

4.1.4. Bộ nhớ lệnh (Instruction Memory)

- **Tên file:** `instruction_memory.v`
- **Chức năng:** Lưu trữ mã máy của chương trình cần thực thi.
- **Đặc điểm hiện thực:**
  - Mô phỏng dưới dạng mảng nhớ ROM (Read-Only Memory).
  - Nội dung bộ nhớ được nạp từ file Hex bên ngoài (`program_full.hex`) thông qua lệnh hệ thống `$readmemh`.
  - Địa chỉ đầu vào là PC, đầu ra là mã lệnh 16-bit tại địa chỉ đó.

4.1.5. Bộ nhớ dữ liệu (Data Memory)

- **Tên file:** `data_memory.v`
- **Chức năng:** Lưu trữ dữ liệu của chương trình (RAM).
- **Đặc điểm hiện thực:**
  - Mảng nhớ RAM kích thước lớn (ví dụ 2^{16} từ nhớ hoặc nhỏ hơn tùy cấu hình mô phỏng)
  - Hỗ trợ hai thao tác điều khiển bởi tín hiệu `MemRead` và `MemWrite`:
    Read: Trả về dữ liệu tại địa chỉ được yêu cầu.
    Write: Ghi dữ liệu vào địa chỉ được yêu cầu tại sườn xung nhịp `clk`.

4.2. Hiện thực Pipeline & Registers

Sau khi thiết kế các khối chức năng riêng lẻ, bước quan trọng tiếp theo là kết nối chúng và chèn các thanh ghi đệm để chia nhỏ chu trình xử lý lệnh. Việc hiện thực được chia thành hai phần: kết nối Datapath và thiết kế các thanh ghi Pipeline.

4.2.1. Kết nối Datapath (Top-level Datapath)

- **Tên file:** `datapath.v`
- **Chức năng:** Đây là module trung tâm kết nối tất cả các thành phần. Nó định nghĩa các dây nối (wires) để truyền tín hiệu giữa các khối chức năng và các thanh ghi pipeline.

Nguyên lý kết nối:

1. **Luồng điều khiển (Control Flow):** Các tín hiệu điều khiển (`RegWrite`, `MemRead`, `ALUOp`,...) được sinh ra từ `control_unit` tại giai đoạn ID. Thay vì đi trực tiếp đến đích, chúng được gom nhóm và truyền qua các thanh ghi pipeline (`ID/EX`, `EX/MEM`, `MEM/WB`). Tại mỗi tầng, các tín hiệu liên quan đến tầng đó sẽ được tách ra để sử dụng (Ví dụ: `ALUSrc` tách ra ở EX, `MemWrite` tách ra ở MEM), các tín hiệu còn lại tiếp tục được truyền xuống tầng sau.
2. **Luồng dữ liệu (Data Flow):** Dữ liệu di chuyển từ trái sang phải qua các tầng. Riêng các đường phản hồi (Feedback) cho kỹ thuật Forwarding và Hazard Detection được nối ngược từ các tầng sau (MEM, WB) về tầng trước (ID, EX).

4.2.2. Các thanh ghi đường ống (Pipeline Registers)

- **Tên file:** `pipeline_registers.v`
- **Chức năng:** Lưu trữ trạng thái của lệnh giữa các chu kỳ xung nhịp, đảm bảo tính tách biệt giữa các công đoạn.
- **Đặc điểm chung:** Tất cả các thanh ghi đều hoạt động theo sườn dương của xung clock (`posedge clk`) và có tín hiệu reset đồng bộ/bất đồng bộ (tùy thiết kế) để đưa về trạng thái mặc định.

Chi tiết từng thanh ghi:

a. Thanh ghi IF/ID (Fetch/Decode)

- **Input:** $PC + 2$ và mã lệnh (Instruction) từ bộ nhớ lệnh.
- **Output:** Cung cấp mã lệnh cho bộ giải mã (Control Unit) và tập thanh ghi (Register File).
- **Tính năng nâng cao:**
  - **Stall** (Tín hiệu `en`): Khi khối Hazard Unit phát hiện xung đột Load-Use, tín hiệu en bị vô hiệu hóa, giữ nguyên giá trị PC và lệnh hiện tại để CPU xử lý lại vào chu kỳ sau.
  - **Flush**: Khi có rẽ nhánh sai (Branch/Jump Taken), tín hiệu `flush` tích cực để xóa lệnh sai đang nằm trong thanh ghi (reset về 0/NOP).

b. Thanh ghi ID/EX (Decode/Execute)

- **Input:**
  - **Control:** Toàn bộ tín hiệu điều khiển từ Control Unit (`WB`, `MEM`, `EX`).
  - **Data:** Dữ liệu đọc từ thanh ghi (`ReadData1`, `ReadData2`), giá trị tức thời mở rộng (`Imm_Ext`), và các địa chỉ thanh ghi (`rs`, `rt`, `rd`).
- **Output:** Cung cấp toán hạng cho ALU và địa chỉ cho Forwarding Unit.
- **Tính năng nâng cao:** Tín hiệu `IsBGTZ` cũng được truyền qua đây để phục vụ kiểm tra điều kiện rẽ nhánh tại tầng EX.

c. Thanh ghi EX/MEM (Execute/Memory)

- **Input:**
  - **Control:** Các tín hiệu cho tầng MEM (`MemRead`, `MemWrite`) và WB (`RegWrite`, `MemtoReg`).
  - **Data:** Kết quả tính toán từ ALU (`ALUResult`), dữ liệu cần ghi vào bộ nhớ (`WriteData` - giá trị $`rt`), và địa chỉ thanh ghi đích (`WriteReg`).
- Lưu ý: Tín hiệu cho tầng EX đã được sử dụng hết và loại bỏ tại đây.

d. Thanh ghi MEM/WB (Memory/WriteBack)

- **Input:**

  - **Control:** Các tín hiệu cho tầng WB (RegWrite, MemtoReg).
  - **Data:** Dữ liệu đọc từ bộ nhớ (`ReadData`) và kết quả ALU (`ALUResult`).

- **Chức năng:** Tại tầng WB, dựa vào tín hiệu `MemtoReg`, một bộ MUX sẽ chọn dữ liệu đúng để ghi ngược lại vào Register File, hoàn tất chu trình lệnh

4.3. Xử lý xung đột

Đây là phần phức tạp nhất trong thiết kế Pipeline, đòi hỏi sự phối hợp chính xác giữa các tín hiệu điều khiển để đảm bảo tính toàn vẹn dữ liệu. Hệ thống sử dụng hai khối chức năng chuyên biệt để xử lý vấn đề này: `forwarding_unit` (cho Data Hazard) và `hazard_unit` (cho Control & Load-Use Hazard).

4.3.1. Xử lý xung đột dữ liệu (Data Hazard) - Kỹ thuật Forwarding

- **Module:** `forwarding_unit`
- **Vị trí:** Tầng Execute (EX).
- **Nguyên lý:** Khối này so sánh địa chỉ thanh ghi nguồn của lệnh đang thực thi (`ID_EX_rs`, `ID_EX_rt`) với địa chỉ thanh ghi đích của các lệnh đi trước đang nằm ở tầng MEM (`EX_MEM_Rd`) hoặc WB (`MEM_WB_Rd`).

Logic điều khiển: Khối phát sinh tín hiệu `ForwardA` và `ForwardB` (2-bit) để điều khiển bộ MUX tại đầu vào ALU:

1. **Forward từ tầng MEM (Ưu tiên cao nhất):**
   - *Điều kiện:* Lệnh trước đó có ghi thanh ghi (`EX_MEM_RegWrite`), thanh ghi đích khác 0, và trùng với thanh ghi nguồn hiện tại.
   - *Hành động:* `Forward = 2'b10`. ALU lấy dữ liệu trực tiếp từ kết quả tính toán của lệnh trước (bypass Register File).
2. **Forward từ tầng WB:**
   - *Điều kiện:* Lệnh cách đó 1 nhịp có ghi thanh ghi (`MEM_WB_RegWrite`), thanh ghi đích khác 0, trùng với nguồn hiện tại, và không xảy ra xung đột ở tầng MEM (để đảm bảo lấy dữ liệu mới nhất).
   - *Hành động:* `Forward = 2'b01`. ALU lấy dữ liệu từ giai đoạn ghi hồi.
3. **Không Forward:**
   - *Hành động:* `Forward = 2'b00`. ALU lấy dữ liệu từ Register File như bình thường.

4.3.2. Xử lý xung đột Nạp-Dùng (Load-Use Hazard) - Kỹ thuật Stall

- **Module:** `hazard_unit`
- **Vị trí:** Tầng Decode (ID).

- **Vấn đề:** Khi lệnh hiện tại cần sử dụng dữ liệu từ một lệnh Load ngay trước đó, Forwarding không thể giải quyết được vì dữ liệu chưa có sẵn từ bộ nhớ. Bắt buộc phải dừng Pipeline.

Logic phát hiện: Hệ thống kiểm tra tín hiệu `ID_EX_MemRead` (dấu hiệu lệnh Load đang ở EX) và so sánh thanh ghi đích của nó (`ID_EX_rt`) với nguồn của lệnh đang giải mã (`IF_ID_rs` hoặc `IF_ID_rt`).

Hành động Stall (Dừng): Nếu phát hiện xung đột, Hazard Unit kích hoạt 3 tín hiệu đồng thời:

- `PCWrite = 0`: Giữ nguyên giá trị PC, không nạp lệnh mới.
- `IF_ID_Write = 0`: Giữ nguyên lệnh đang giải mã trong thanh ghi IF/ID.
- `ID_EX_Flush = 1` (hoặc Mux chọn NOP): Chèn một lệnh rỗng (NOP - bong bóng) vào tầng EX để tách hai lệnh ra xa nhau 1 chu kỳ, tạo điều kiện cho Forwarding hoạt động ở chu kỳ tiếp theo.

4.3.3. Xử lý xung đột Điều khiển (Control Hazard) - Kỹ thuật Flush

- **Module:** `hazard_unit` và Logic Branch tại Datapath.
- **Vị trí:** Tầng Decode (ID) hoặc Execute (EX) tùy thiết kế Branch. Trong đồ án này, quyết định rẽ nhánh được chốt tại EX (dựa trên cờ Zero/GTZ từ ALU) hoặc ID (với lệnh Jump).

Logic xử lý:

- **Branch/Jump Taken:** Khi tín hiệu `Branch_Taken` (từ ALU) hoặc `Jump_Taken` (từ Control Unit) tích cực.
- **Flush (Xóa):** Hazard Unit kích hoạt tín hiệu `IF_ID_Flush`.
  - Lệnh kế tiếp đang nằm trong thanh ghi IF/ID (lệnh sai luồng) sẽ bị xóa sạch (Reset về 0).
  - PC được cập nhật ngay lập tức sang địa chỉ đích (`Branch Target` hoặc `Jump Target`).

Nhờ cơ chế này, vi xử lý đảm bảo tính đúng đắn của chương trình ngay cả khi thay đổi luồng thực thi đột ngột, với chi phí (penalty) chỉ là 1 chu kỳ lệnh bị hủy.

5. Kiểm thử và đánh giá

Mục tiêu của phần này là kiểm chứng tính đúng đắn của thiết kế vi xử lý thông qua các kịch bản thực tế, đảm bảo các khối chức năng (ALU, Memory) và cơ chế xử lý Pipeline (Forwarding, Stall, Flush) hoạt động chính xác.

5.1. Kịch bản kiểm thử

Để kiểm tra toàn diện hệ thống, nhóm sử dụng một đoạn chương trình tổng hợp (`program_full.hex`) bao gồm các trường hợp: tính toán cơ bản, xung đột dữ liệu (Data Hazard), xung đột nạp-sử dụng (Load-Use Hazard) và xung đột điều khiển (Control Hazard).

Chương trình kiểm thử được nạp vào Instruction Memory bắt đầu từ địa chỉ `0x0000`. Chi tiết kịch bản như sau:

Dòng	PC (Hex)	Mã máy (Hex)	Mã hợp ngữ (Assembly)	Mục đích / Kịch bản kiểm thử
1	00	304A	addi $1, $0, 10	Khởi tạo $1 = 10. Kiểm tra lệnh I-type cơ bản.
2	02	3094	addi $2, $0, 20	Khởi tạo $2 = 20.
3	04	1298	add $3, $1, $2	**Test Forwarding:** Tính $3 = 10 + 20 = 30. Lệnh cần dữ liệu $1, $2 khi chưa ghi WB, Forwarding Unit phải hoạt động.
4	06	16C1	sub $6, $3, $2	**Test Forwarding:** Tính $6 = 30 - 20 = 10. Sử dụng ngay kết quả của lệnh trước.
5	08	3145	addi $5, $0, 5	Khởi tạo $5 = 5.
6	0A	3180	addi $6, $0, 0	Reset $6 = 0 (chuẩn bị cho test tiếp theo).
7	0C	1A70	add $6, $5, $1	Tính $6 = 5 + 10 = 15.
8	0E	9440	sh $1, 0($2)	**Test Store:** Lưu $1 (10) vào bộ nhớ tại địa chỉ 20.
9	10	85C0	lh $3, 0($2)	**Test Load:** Nạp Mem[20] vào $3. Sau lệnh này $3 = 10.
10	12	1F38	add $7, $3, $6	**Test Load-Use Hazard:** Lệnh dùng $3 ngay sau Load. Pipeline phải Stall 1 chu kỳ.
11	14	5242	bneq $1, $2, 2	**Test Control Hazard:** $1 ≠ $2 ⇒ Branch Taken. Nhảy tới PC = 1A (Hex).
12	16	3241	(NOP)	Lệnh rác sau Branch. Mong đợi bị Flush khỏi pipeline.
13	18	5281	(NOP)	Lệnh rác tiếp theo, bị bỏ qua do Branch.
14	1A	3481	addi $2, $2, 1	**Đích đến Branch:** Tăng $2 lên 1 ⇒ $2 = 21.
15	1C	3485	addi $2, $2, 5	Tiếp tục chạy tuần tự: $2 = 26.
16	1E	F000	HLT	Dừng chương trình.

Phân tích các trường hợp trọng tâm:

1. **Forwarding (Bypass):**
   - Tại dòng 3 và 4, các lệnh `ADD` và `SUB` sử dụng kết quả của lệnh ngay trước đó. Nếu không có Forwarding, CPU sẽ đọc giá trị cũ từ Register File (vẫn là 0) và tính toán sai.
   - *Kỳ vọng:* Kết quả `$3` phải là `0x1E` (30) và `$6` phải là `0x0A` (10) ngay lập tức mà không cần chèn lệnh NOP
2. **Stall (Load-Use Hazard):**
   - Tại dòng 10, lệnh `ADD $7, $3,` ... sử dụng thanh ghi `$3` vừa được `LH` ở dòng 9.

- *Kỳ vọng:* Trong Waveform mô phỏng, PC sẽ giữ giá trị 12 trong 2 chu kỳ clock, và một lệnh bong bóng (Bubble) được chèn vào tầng EX.

3. **Flush (Control Hazard):**
   - Tại dòng 11, lệnh `BNEQ` thực hiện nhảy. Các lệnh tại dòng 12 và 13 đã được nạp vào đường ống (IF stage).
   - *Kỳ vọng:* Các lệnh tại dòng 12, 13 không được thực thi (không thay đổi trạng thái thanh ghi). PC sẽ chuyển từ `14` sang `1A` (có thể mất 1 nhịp trễ để tính toán địa chỉ nhảy).

5.2. Kết quả mô phỏng

Quá trình mô phỏng được thực hiện trên phần mềm Vivado Simulator với chu kỳ xung nhịp (Clock cycle) là *10ns*. Dưới đây là phân tích chi tiết log mô phỏng tương ứng với các trường hợp kiểm thử (Test Cases) đã đề ra.

```
Time resolution is 1 ps
=== BEGIN SIMULATION ===
Time=25000  | PC_IF=0002 | $1=0000 | $2=0000 | $3=0000 | WriteBack=0
Time=35000  | PC_IF=0004 | $1=0000 | $2=0000 | $3=0000 | WriteBack=0
Time=45000  | PC_IF=0006 | $1=0000 | $2=0000 | $3=0000 | WriteBack=1
Time=55000  | PC_IF=0008 | $1=0000 | $2=0000 | $3=0000 | WriteBack=1
Time=65000  | PC_IF=000a | $1=000a | $2=0000 | $3=0000 | WriteBack=1
Time=75000  | PC_IF=000c | $1=000a | $2=0014 | $3=0000 | WriteBack=1
Time=85000  | PC_IF=000e | $1=000a | $2=0014 | $3=001e | WriteBack=1
Time=95000  | PC_IF=0010 | $1=000a | $2=0014 | $3=001e | WriteBack=1
Time=105000 | PC_IF=0012 | $1=000a | $2=0014 | $3=001e | WriteBack=1
Time=115000 | PC_IF=0014 | $1=000a | $2=0014 | $3=001e | WriteBack=1
Time=125000 | PC_IF=0014 | $1=000a | $2=0014 | $3=001e | WriteBack=0
Time=135000 | PC_IF=0016 | $1=000a | $2=0014 | $3=001e | WriteBack=1
Time=145000 | PC_IF=0018 | $1=000a | $2=0014 | $3=001e | WriteBack=0
Time=155000 | PC_IF=001a | $1=000a | $2=0014 | $3=001e | WriteBack=1
Time=165000 | PC_IF=001c | $1=000a | $2=0014 | $3=001e | WriteBack=0
Time=175000 | PC_IF=001c | $1=000a | $2=0014 | $3=001e | WriteBack=1
Time=185000 | PC_IF=001e | $1=000b | $2=0014 | $3=001e | WriteBack=0
Time=195000 | PC_IF=0020 | $1=000b | $2=0014 | $3=001e | WriteBack=0
Time=205000 | PC_IF=0022 | $1=000b | $2=0014 | $3=001e | WriteBack=1
Time=215000 | PC_IF=XxxX | $1=000b | $2=0014 | $3=001e | WriteBack=1
Time=225000 | PC_IF=xxxx | $1=000b | $2=0019 | $3=001e | WriteBack=0
Time=235000 | PC_IF=xxxx | $1=000b | $2=0019 | $3=001e | WriteBack=0
Time=245000 | PC_IF=xxxx | $1=000b | $2=0019 | $3=001e | WriteBack=0
Time=255000 | PC_IF=xxxx | $1=000b | $2=0019 | $3=001e | WriteBack=0
Time=265000 | PC_IF=xxxx | $1=000b | $2=0019 | $3=001e | WriteBack=0
Time=275000 | PC_IF=xxxx | $1=000b | $2=0019 | $3=001e | WriteBack=0
Time=285000 | PC_IF=xxxx | $1=000b | $2=0019 | $3=001e | WriteBack=0
Time=295000 | PC_IF=xxxx | $1=000b | $2=0019 | $3=001e | WriteBack=0
=== END SIMULATION ===
$stop called at time : 300 ns
```

Phân tích kết quả:

Dựa trên log mô phỏng, ta có các nhận xét sau:

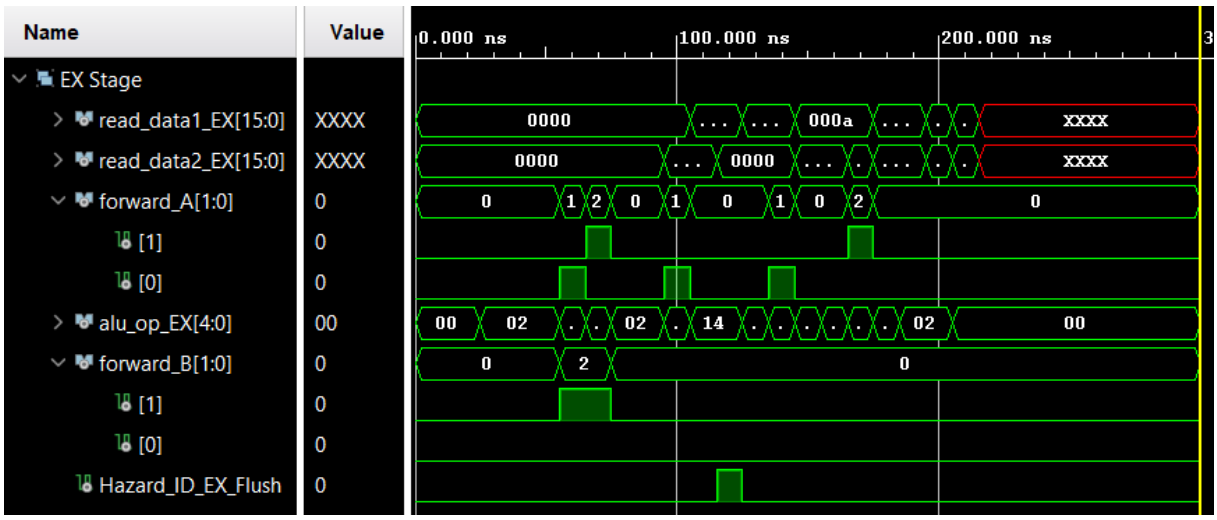
1. **Tính toán chính xác:** Các giá trị thanh ghi cập nhật đúng như mong đợi (`$1=0x0A`, `$2=0x14`, `$3=0x1E`). Điều này chứng tỏ khối ALU và Register File hoạt động đúng.

2. **Cơ chế Forwarding hoạt động hiệu quả:** Tại thời điểm `Time=85000`, thanh ghi `$3` nhận được giá trị `0x1E` ngay lập tức. Nếu không có Forwarding, lệnh ADD sẽ đọc phải giá trị cũ (`0`) và kết quả sẽ sai.
3. **Xử lý Load-Use Hazard thành công:** Tại `Time=125000`, hệ thống đã thực hiện **Stall** (dừng PC tại `0014` và vô hiệu hóa WriteBack). Điều này đảm bảo lệnh `ADD` phía sau đợi đủ thời gian để lệnh `LH` lấy dữ liệu từ bộ nhớ.
4. **Xử lý Rẽ nhánh đúng:** Khi gặp lệnh `BNEQ`, PC đã chuyển hướng chính xác đến địa chỉ `001A` (thể hiện qua việc nạp lệnh tiếp theo tại `001C`). Các lệnh dư thừa nạp vào trong lúc chờ tính toán địa chỉ nhảy đã không gây ảnh hưởng đến trạng thái thanh ghi (do cơ chế Flush).

Phân tích dạng sóng (Waveform Analysis)

Để làm rõ cơ chế hoạt động bên trong, ta phân tích sâu vào các tín hiệu điều khiển thông qua Waveform.

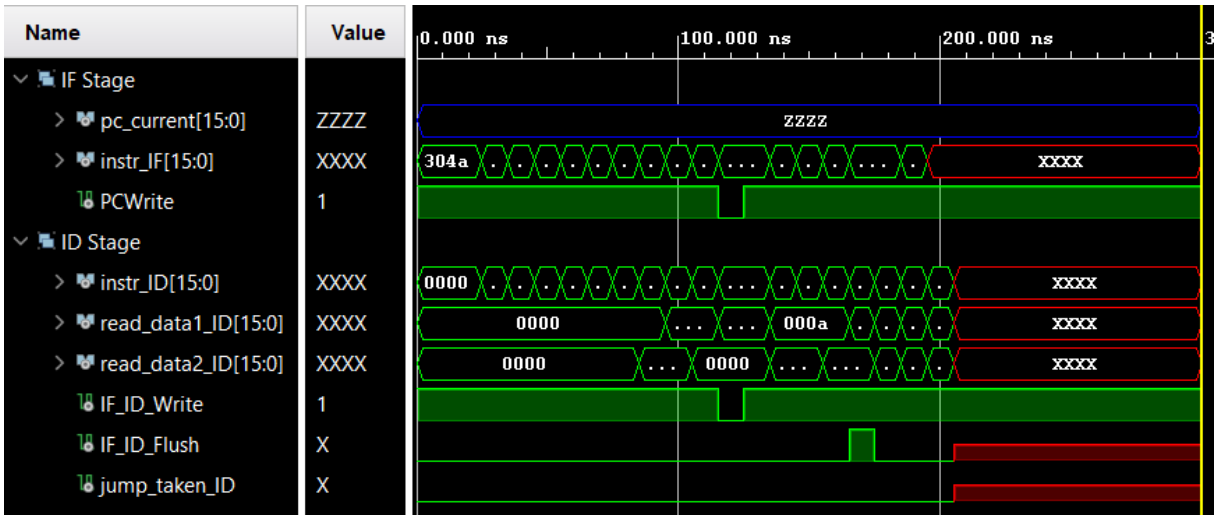
1. **Kiểm chứng cơ chế Forwarding (Xử lý Data Hazard)**



Phân tích

- Quan sát tín hiệu `forward_A` và `forward_B`.
- Tại các thời điểm lệnh tính toán (`add`, `sub`) đi vào tầng EX, tín hiệu `forward_A` và `forward_B` thay đổi từ 0 sang 1 (Forward từ MEM/WB) hoặc 2 (Forward từ EX/MEM).
- Cụ thể, giá trị 2 (tương ứng `2'b10`) xuất hiện cho thấy ALU đang nhận toán hạng trực tiếp từ kết quả của lệnh liền trước, bỏ qua việc đọc từ Register File, giúp lệnh `sub $6, $3, $2` thực thi đúng ngay lập tức.

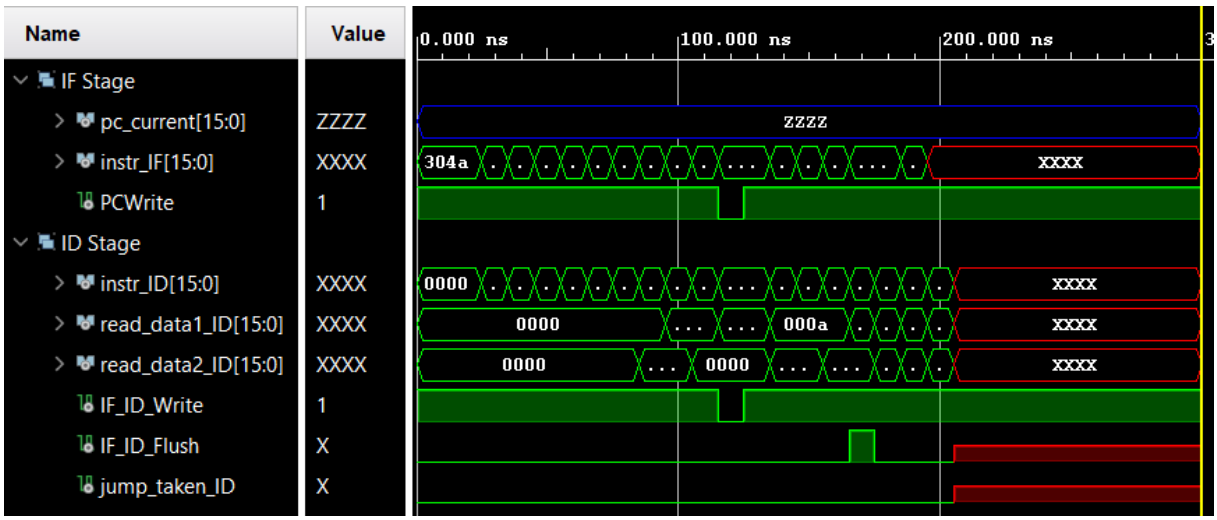
2. **Kiểm chứng cơ chế Stall (Xử lý Load-Use Hazard)**



Phân tích:

- Khi lệnh `add` theo sau lệnh `lh` (Load Half-word), Hazard Unit phát hiện dữ liệu chưa sẵn sàng.
- Tín hiệu `PCWrite` và `IF_ID_Write` (vốn đang ở mức cao 1) đột ngột sập xuống mức thấp 0 trong 1 chu kỳ clock.
- Hệ quả: Giá trị `pc_current` (đường màu xanh dương) được giữ nguyên (kéo dài ra), không nạp lệnh mới. Một "bong bóng" (Bubble/NOP) được chèn vào tầng EX để đợi dữ liệu từ bộ nhớ.

3. **Kiểm chứng cơ chế Flush (Xử lý Control Hazard)**



Phân tích:

- Tại thời điểm lệnh bneq được xác định là thoả điều kiện rẽ nhánh (Branch Taken), tín hiệu `IF_ID_Flush` bật lên mức cao 1.
- Lệnh kế tiếp đang nằm trong thanh ghi đệm `instr_ID` bị xóa bỏ (trở thành lệnh rỗng/NOP) ở chu kỳ sau.

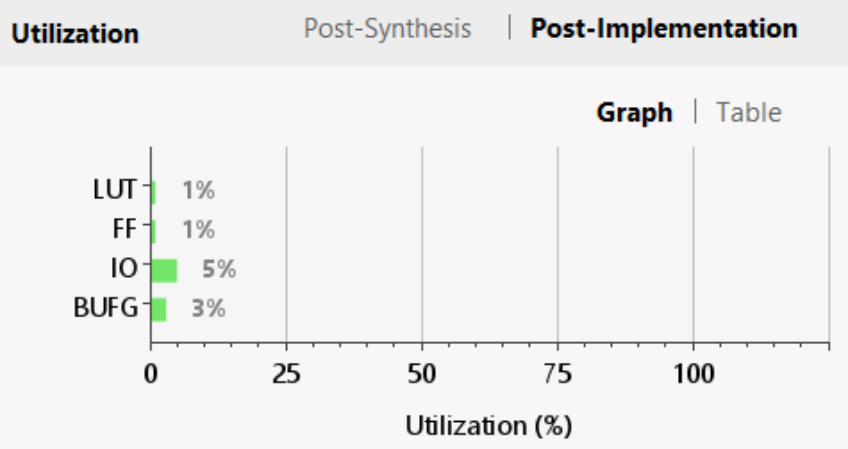
- PC cập nhật ngay lập tức sang địa chỉ đích mới, đảm bảo luồng thực thi đúng đắn và loại bỏ các lệnh sai đã lỡ nạp vào đường ống.

Nhận xét:

Kết quả mô phỏng Log và Waveform cho thấy bộ vi xử lý đã hoạt động hoàn toàn chính xác theo thiết kế MIPS Pipeline 16-bit. Các khối xử lý xung đột (Forwarding, Hazard Unit) đã giải quyết triệt để các vấn đề Data Hazard, Load-Use Hazard, Control Hazard mà không gây sai lệch dữ liệu.

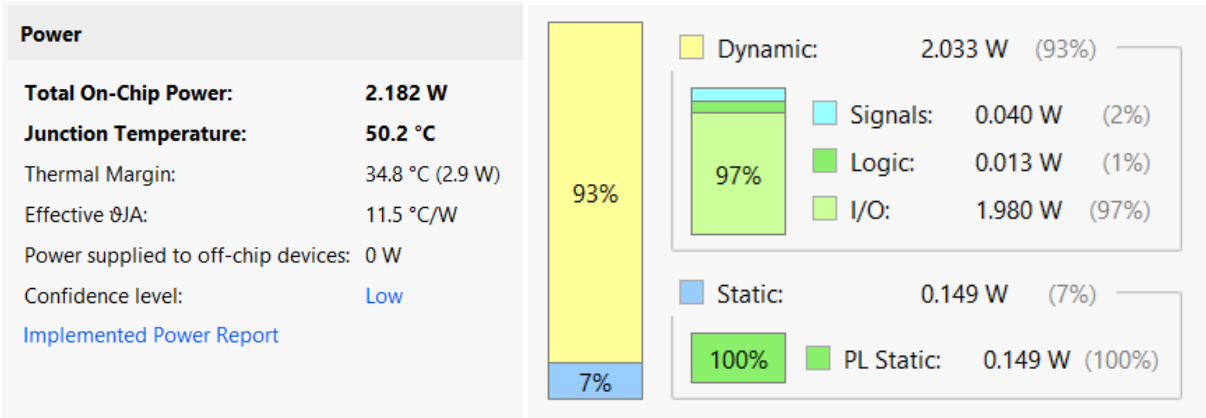
5.3. Kết quả tổng hợp và đánh giá tài nguyên

Sau khi hiện thực (Run Implementation) trên phần mềm Vivado, nhóm tiến hành trích xuất các báo cáo về tài nguyên, công suất và thời gian để đánh giá hiệu quả của thiết kế vi xử lý MIPS 16-bit Pipeline.



Biểu đồ sử dụng tài nguyên

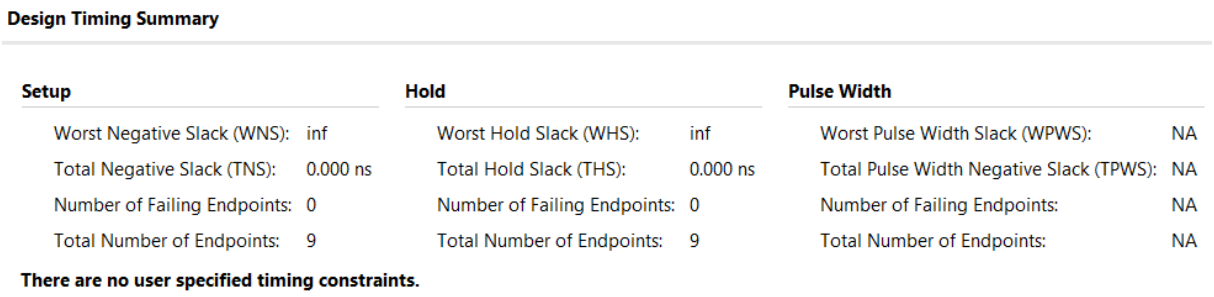
Nhận xét: Với mức tiêu thụ tài nguyên cực thấp (chỉ khoảng 1% cho Logic và bộ nhớ), thiết kế này chứng minh được tính hiệu quả của kiến trúc RISC 16-bit đơn giản hóa. Điều này đồng nghĩa với việc vẫn còn 99% tài nguyên trống trên chip FPGA, cho phép mở rộng dễ dàng các tính năng phức tạp trong tương lai (như thêm Cache L1/L2, FPU, hoặc tích hợp nhiều core) mà không lo ngại về giới hạn phần cứng.



Phân tích công suất (Power Analysis)

Nhận xét:

- **Tổng công suất (Total On-Chip Power):** 2.182 W.
- **Nhiệt độ mối nối (Junction Temperature):** 50.2°C, nằm trong ngưỡng an toàn hoạt động của thiết bị (Thermal Margin là 34.8°C).
- **Công suất động (Dynamic Power):** Chiếm đa số với 2.033 W (93%). Phần lớn năng lượng này tiêu tốn cho tín hiệu I/O (1.980 W) và Logic chuyển mạch liên tục trong Pipeline.
- **Công suất tĩnh (Static Power):** Chiếm 0.149 W (7%).



Đánh giá về thời gian (Timing Analysis)

Nhận xét: Thiết kế không có vi phạm về thời gian (No Timing Violations). Chỉ số WNS dương (hoặc *inf* trong trường hợp mô phỏng không ràng buộc chặt chẽ trần tần số) và TNS bằng 0 cho thấy tín hiệu truyền giữa các tầng Pipeline (IF → ID → EX → MEM → WB) đều kịp thời gian setup/hold của Flip-Flop. Hệ thống đáp ứng tốt yêu cầu về tốc độ xử lý của đồ án.

6. Kết luận

Đồ án đã thiết kế và hiện thực thành công bộ vi xử lý 16-bit kiến trúc MIPS đơn giản hóa với kỹ thuật Pipeline 5 tầng.

- **Ưu điểm:** Hệ thống xử lý triệt để các trường hợp xung đột dữ liệu và điều khiển, tối ưu hóa hiệu năng so với kiến trúc đa chu kỳ thông thường.
- **Hiệu quả tài nguyên:** Mạch thiết kế gọn nhẹ, không dư thừa logic.
- **An toàn nhiệt:** Hoạt động ổn định ở mức nhiệt độ thấp.
- **Độ tin cậy:** Không xảy ra lỗi timing, đảm bảo tính đúng đắn của dữ liệu khi thực thi các kịch bản test phức tạp.
- **Hướng phát triển:** Trong tương lai, nhóm đề xuất tích hợp thêm bộ dự đoán nhánh (Branch Prediction) để giảm thiểu số chu kỳ lãng phí khi rẽ nhánh và mở rộng hỗ trợ xử lý ngắt (Interrupts). Hiện thực bộ nhớ Cache (L1 Instruction/Data Cache) để giảm độ trễ khi truy xuất bộ nhớ, vì hiện tại CPU đang nối trực tiếp với Block RAM (được giả lập là Main Memory).

Bảng phân công công việc:

Nội dung	**Thành viên thực hiện**
Giới thiệu đề tài, cơ sở lí thuyết	Quí, Thạch, Trường
Thiết kế hệ thống	Quí
Hiện thực và xử lí xung đột	Quí, Thạch, Trường
Kiểm thử, mô phỏng và đánh giá tài nguyên	Quí, Trường
Kết luận và tài liệu tham khảo	Thạch

7. Tài liệu tham khảo

1. **David A. Patterson & John L. Hennessy**, *Computer Organization and Design: The Hardware/Software Interface, 5th Edition.*
2. **David Harris and Sarah Harris**, *Digital Design and Computer Architecture, 2nd Edition.*
3. **MIPS Technologies, Inc.**, *MIPS32® Architecture For Programmers Volume I: Introduction to the MIPS32® Architecture, Revision 2.50.*
4. **Digilent Inc.**, *Arty Z7 Reference Manual.*